

DATA266 LAB 1: LANGUAGE MODELLING, SENTIMENT CLASSIFICATION AND UNPAIRED IMAGE TRANSLATION

Anees Saheba Guddi (SJSU ID 018205330) and Yashashree Shinde (SJSU ID 019134349)

Team 8, DATA266, San Jose State University

Repository: <https://github.com/aneessaheba/DATA-266-Lab-1>

Abstract—This report covers three machine learning systems built from scratch by a team of two, with each member independently designing, training and evaluating their own model for every task so that all results appear twice. Task 1 is a character level GPT trained on TinyStories, where the two models reach validation bits per character of 1.0875 and 1.0210. Task 2 is Yelp Polarity sentiment classification with three classifiers per member, where the best model reaches 95.02 percent test accuracy. Task 3 is a CycleGAN for unpaired translation between Monet paintings and photographs, where the two runs reach photo to Monet FID of 86.59 and 92.39 and Kaggle public scores of -52.8390 and -54.6990. Three findings emerged only from comparing the two independent runs: the language model with the better validation loss produces the more repetitive text, mean pooling beats max pooling as a sentiment baseline by 3.6 points, and the CycleGAN identity weight trades style fidelity against cycle reconstruction with each member winning the side their own weight favoured. All reported numbers are traceable to committed files.

Index Terms—transformer, causal self attention, character level language model, sentiment classification, recurrent networks, generative adversarial networks, unpaired image translation, cycle consistency, reproducibility.

TEAM OWNERSHIP STATEMENT

Both members independently designed, coded, trained and evaluated their own models for all three tasks. Every number in this report therefore exists twice, once per member, produced by two separate runs on two different machines.

- **Anees Saheba Guddi** built the four block character level GPT for Task 1, the max pooling, LSTM and attention pooling classifiers for Task 2, and the six block CycleGAN for Task 3. He also set up the repository, the branching model, the shared data download script and the one command smoke test.
- **Yashashree Shinde** built the six block character level GPT for Task 1, the mean pooling, convolutional and bidirectional GRU classifiers for Task 2, and the nine block CycleGAN for Task 3.
- Both members submitted independently to the Kaggle class competition.
- The comparison tables, the joint analyses and this report were written together from both sets of results.

I. TASK 1: CHARACTER LEVEL LANGUAGE MODEL

A. Objective

Build a GPT style autoregressive language model from scratch on TinyStories, train it for at least ten epochs, report the full metric list, and analyse three real failure cases from generated text. No prebuilt transformer or attention modules were permitted.

Both members wrote multi head causal self attention with explicit matrix multiplications and verified numerically that no position can attend to a future position.

B. Member 1: Anees Saheba Guddi

1) Architecture

TABLE I

Component	Value
Transformer blocks	4
Embedding width	256
Attention heads	4
Head dimension	64
Feedforward width	1024
Activation	GELU
Normalisation	pre layer norm
Positional embedding	learned
Context length	128 characters
Dropout	0.10
Vocabulary	91 characters
Parameters	3,238,912

Parameter breakdown, which reconciles exactly:

- Token embedding: 23,296
- Positional embedding: 32,768
- Four transformer blocks at 789,760 each: 3,159,040
- Final layer norm: 512
- Language model head: 23,296
- **Total: 3,238,912**

2) Hyperparameters and reasons

- **Seed 5330**, the last four digits of his student ID, so initialisation, the train and validation split and the shuffling are his own.
- **Optimiser AdamW**, with weight decay applied to matmul weights only, not to layer norms or embeddings.
- **Learning rate $3e-4$** with 1000 warmup steps, then cosine decay to ten percent of peak.
- **Gradient clipping at 1.0**, which the gradient norm maximum of 6.49 shows was doing real work.
- **Batch size 64**, ten epochs, 15,620 optimiser steps in total.

3) Data preprocessing

The source text contained mojibake, UTF-8 that had been decoded as cp1252. He found and repaired it with an explicit byte sequence map.

- Before repair: vocabulary of 101 characters, including artifacts such as a three byte sequence standing in for a single quotation mark.
- After repair: vocabulary of 91 characters, zero markers remaining.
- The first repair attempt silently failed because it encoded as Latin 1, where the characters involved are cp1252 only, so the encode raised and the function returned the text unchanged. The bug was caught because the vocabulary size did not drop.

4) Results

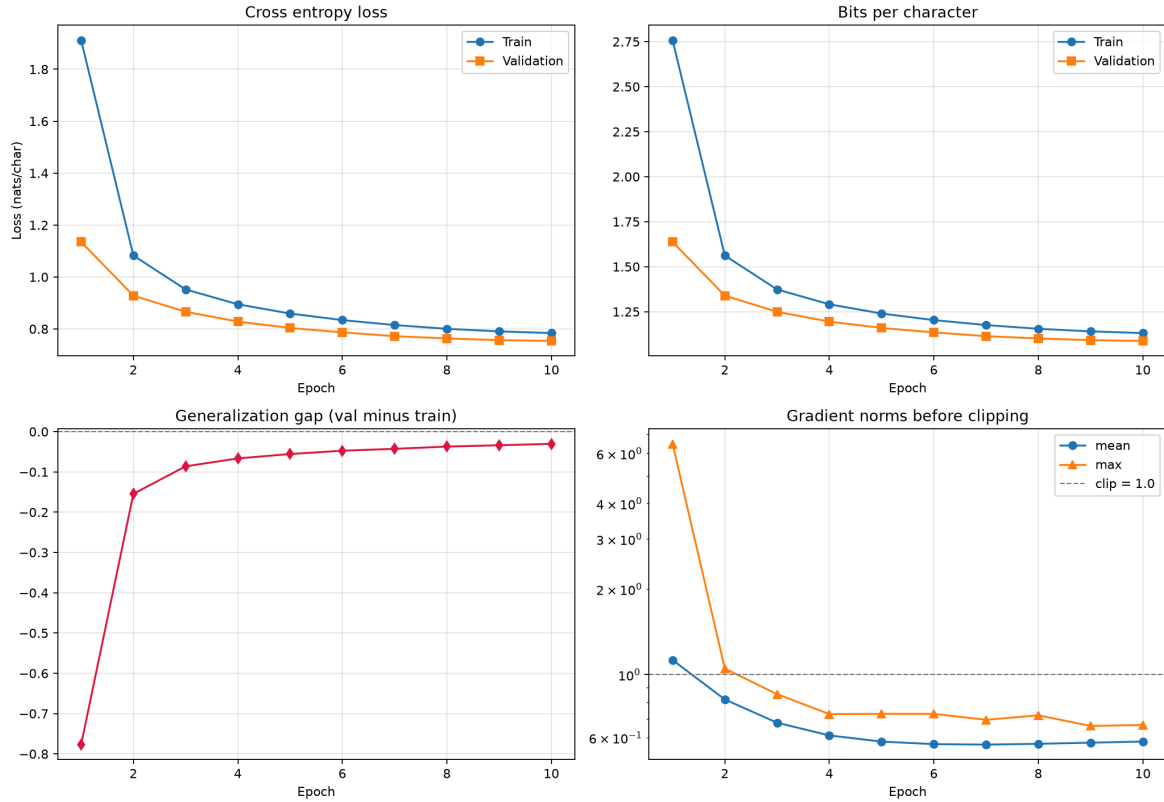


Fig. 1. Training and validation loss curves over ten epochs, Anees. Validation loss sits at or below training loss throughout, giving a generalisation gap of -0.0303.

TABLE II

Metric	Value
Training cross entropy loss	0.7841
Validation cross entropy loss	0.7538
Training perplexity	2.1904
Validation perplexity	2.1251
Validation bits per character	1.0875
Top 1 next character accuracy	0.7607
Generalisation gap	-0.0303
Gradient norm mean	0.6679
Gradient norm median	0.5886
Gradient norm maximum	6.4868
Loss spikes	0
NaN count	0
Training throughput	37,528 tokens per second
Generation throughput	233 tokens per second
Peak memory	1.104 GB
Total training time	3,557 s
Device	Apple M4, MPS

Generation diversity, averaged across twelve samples:

TABLE III

Metric	Value
Distinct 1	0.0467
Distinct 2	0.2703
Distinct 3	0.5412
Repeated 4gram rate, all decodings	0.2943
Repeated 4gram rate, greedy only	0.4783

Decoding comparison across his own samples:

TABLE IV

Decoding	Repeated 4gram rate	Distinct 3
Greedy	0.249	0.658
Temperature 0.5	0.186	0.697
Temperature 0.8	0.132	0.763
Temperature 1.2	0.117	0.791

5) Failure analysis

Case 1, repetition. Prompt “One day, Tom”, greedy decoding.

One day, Tom went to the park with his mom. He saw a big storm on the ground. He wanted to see what was inside. He wanted to see what was inside.

- The same sentence appears twice, word for word.
- This sample has the worst repetition of the twelve generated, 0.316 against 0.104 for the most diverse.
- Greedy decoding always takes the most probable next character, so the process is deterministic. Once the model has written a sentence and a full stop the hidden state resembles its earlier state, the same continuation becomes most probable again, and nothing can break the loop.
- The model has no memory of having written that sentence, only 128 characters of context.

Case 2, loss of coherence. Prompt “Once upon a time”, greedy decoding.

They saw a big boy who was very sad and angry. They wanted to show her mommy how to be careful when they saw the boy was so happy to have a new friend

- The boy is introduced as very sad and angry, then described as so happy, within one sentence and with nothing in between to explain it.
- The grammar is correct throughout. What fails is state tracking.
- The model predicts from the previous 128 characters only and has no representation of a character in the story or of facts already asserted.
- Nothing in the training objective penalises contradicting something written 40 characters earlier.

Case 3, hallucination and broken grammar. Prompt “Once upon a time”, temperature 1.2.

She loved playing outside and majwming running around. One day, Anna went to the park so much adventure. That day she was even surprised eating. Carefully, here no hourse. She saw her mom radious and ate the pastels.

- Three strings are not English words: majwming, hourse, radious.
- Grammar also breaks down in “went to the park so much adventure”.
- This failure is specific to character level modelling. A word level model can only emit words from its vocabulary; a character level model over 91 characters can reach any letter sequence.

- At temperature 1.2 the logits are divided before the softmax, which flattens the distribution. One unlikely character early in a word pushes the remaining context off the distribution the model learned.

What the three cases show together.

- Cases 1 and 3 are the two ends of one decoding tradeoff. Lowering temperature produces repetition, raising it produces invented words, and no setting removes both because the causes are opposite.
- Case 2 is not on that axis. It appears at every temperature and comes from the context window rather than from decoding.
- With 128 characters of context the model cannot attend to a fact stated earlier than roughly two sentences back. Increasing context length is the first change he would test.

C. Member 2: Yashashree Shinde

1) Architecture

TABLE V

Component	Value
Transformer blocks	6
Embedding width	192
Attention heads	3
Head dimension	64
Feedforward width	768
Activation	GELU
Normalisation	pre layer norm
Positional embedding	learned
Context length	256 characters
Dropout	0.15
Vocabulary	101 characters
Parameters	2,757,605

2) Hyperparameters and reasons

- **Seed 4349**, the last four digits of her student ID, used for weight initialisation and for independently shuffling the train and validation split.
- **Optimiser AdamW**, weight decay 0.01.
- **Learning rate 3e-4** with 1000 warmup steps, then cosine decay.
- **Gradient clipping at 1.0**.
- **Batch size 64**, ten epochs.
- Deliberately narrower and deeper than her teammate's model, with twice the context length, so that the two runs differ along a meaningful axis.

3) Data preprocessing

- Dataset: TinyStories, character level tokenisation.
- Training sequences: 100,000. Validation sequences: 10,000.
- Sequence input length: 256.
- Train and validation split independently shuffled using seed 4349.
- The mojibake present in the source text was not repaired, so her vocabulary of 101 includes the corrupted sequences as if they were real characters.

4) Results

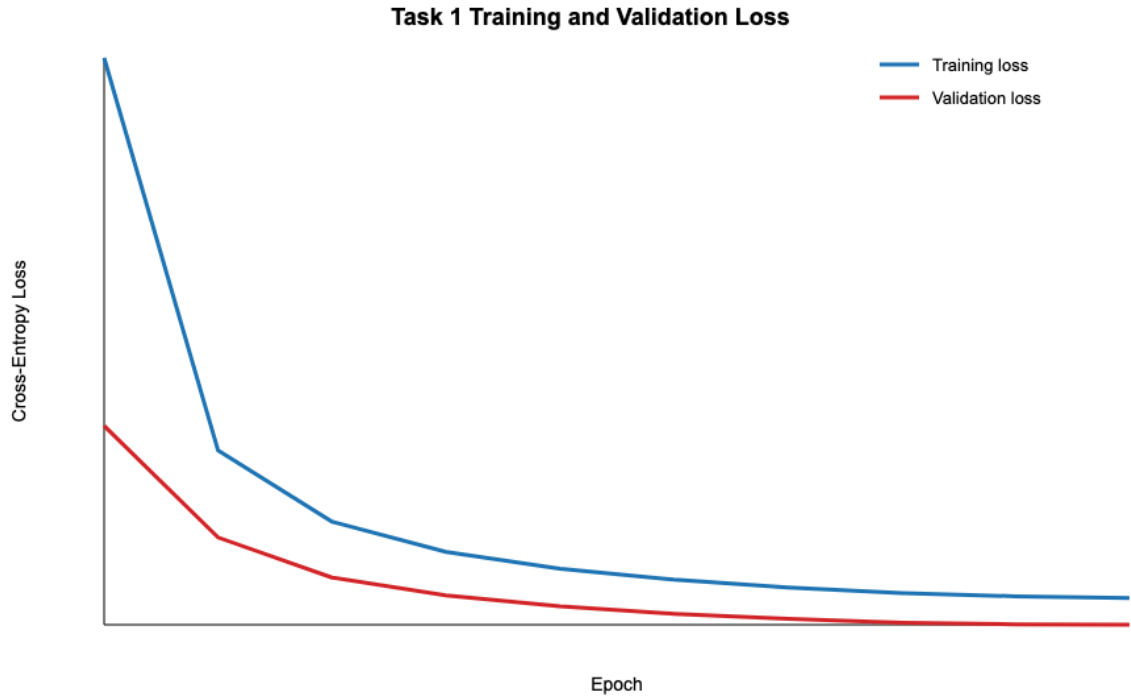


Fig. 2. Training and validation loss curves over ten epochs, Yashashree. The two curves stay together, giving a generalisation gap of -0.0009.

TABLE VI

Metric	Value
Training cross entropy loss	0.7086
Validation cross entropy loss	0.7077
Training perplexity	2.0312
Validation perplexity	2.0293
Training bits per character	1.0223
Validation bits per character	1.0210
Training top 1 accuracy	0.7756
Validation top 1 accuracy	0.7759
Generalisation gap	-0.0009
Average gradient norm	0.6070
Maximum gradient norm	5.4474
Loss spikes	0
NaN count	0
Training throughput	610,618 tokens per second
Generation throughput	351 tokens per second
Peak memory	1.971 GB
Total training time	435 s
Device	NVIDIA RTX 4090

Generation diversity at temperature 0.8:

TABLE VII

Metric	Value
Distinct 1	0.0974
Distinct 2	0.4228
Distinct 3	0.6663
Repeated 4gram rate	0.2310

5) Failure analysis

Case 1, broken grammar and semantic inconsistency.

| *She had a big smile on her house and a special wind.*

- A smile is normally associated with a person, not a house.
- A special wind is not connected clearly to the rest of the sentence.
- The model learned common words and sentence patterns but did not learn to maintain meaningful relationships between objects and actions.

Case 2, loss of coherence and abrupt topic change.

| *Every day, she saw a big tree with lots of money. The pink was a big pretty girl with a playground beautiful pla*

- The text begins with a girl observing a tree, then introduces money, a pink girl and a playground with no logical transition.
- The final phrase is incomplete.
- The model generates locally plausible phrases but cannot maintain a consistent storyline over a longer sequence.

Case 3, repetition and semantic inconsistency.

| *They saw a big tree with a big rock. The tree was filled with a tall green ball on the ground. Lily was so happy that she saw that she wanted to go off to the ground.*

- The model repeats big, tree and ground.
- It describes a tree filled with a ball, and produces an unclear sentence about going off to the ground.
- Frequent character and word patterns are learned without preserving realistic object relationships or grammatical structure.

Her overall observation.

- The model reaches a low validation loss and approximately 77.6 percent next character accuracy while still producing these failures.
- Next character accuracy can be high even when the errors that do occur damage the meaning of a longer story.
- Possible improvements to test: more training data, more model capacity, better context quality, longer training.

D. Side by Side Comparison

TABLE VIII

	Anees	Yashashree
Transformer blocks	4	6
Embedding width	256	192
Attention heads	4	3
Head dimension	64	64
Feedforward width	1024	768
Context length	128	256
Dropout	0.10	0.15
Vocabulary	91	101
Parameters	3,238,912	2,757,605
Optimiser	AdamW	AdamW
Learning rate	3e-4	3e-4
Warmup steps	1000	1000
Schedule	cosine decay	cosine decay
Batch size	64	64
Epochs	10	10
Seed	5330	4349
Validation cross entropy	0.7538	0.7077
Validation perplexity	2.1251	2.0293
Validation bits per character	1.0875	1.0210
Top 1 accuracy	0.7607	0.7759
Generalisation gap	-0.0303	-0.0009
Gradient norm mean	0.6679	0.6070
Gradient norm maximum	6.4868	5.4474
Loss spikes	0	0
NaN count	0	0
Distinct 3 at T 0.8	0.7630	0.6663
Repeated 4gram at T 0.8	0.1320	0.2310
Training throughput	37,528 tok/s	610,618 tok/s
Peak memory	1.104 GB	1.971 GB
Total training time	3,557 s	435 s
Hardware	Apple M4 MPS	RTX 4090

E. Joint Analysis

Strengths.

- Both models learned the TinyStories distribution to a similar degree, around 2.0 to 2.1 perplexity and 76 to 78 percent next character accuracy.
- Neither run produced a single loss spike or non finite step across ten epochs.
- Both generalisation gaps are negative, so validation loss sits at or below training loss and neither model overfitted.
- Causal masking was verified numerically in both notebooks.

Weaknesses.

- Both models fail at state tracking over spans longer than roughly two sentences, and both failure analyses independently identified this.
- Both produce repetition under greedy decoding and invented words at high temperature.
- Neither model was trained long enough to close the gap between next character accuracy and story level coherence.

What the comparison shows.

- Yashashree reaches a better bits per character, 1.0210 against 1.0875, with 15 percent fewer parameters.
- The two candidate explanations are her greater depth and her longer context. Anees's failure analysis, written before the comparison, named the 128 character context as the binding constraint. Her result is consistent with that, but depth and context changed together, so this is evidence rather than proof.
- The result that surprised the team: the model with the better validation loss produces the more repetitive text. At the same temperature Anees's repeated 4gram rate is 0.1320 against 0.2310, and his distinct 3 is 0.7630 against 0.6663.
- Perplexity measures how well a model predicts the next character given a true prefix. Repetition is a property of feeding the model its own output, which perplexity never observes. These are different questions and the ranking does not have to agree.

Limitations.

- The two vocabularies are not the same set, 91 against 101, because only one member repaired the mojibake in the source text. Bits per character is still comparable since it is normalised per character, but a few of the characters in her run are artifacts. If anything this strengthens her result.
- Training times are not comparable, 3,557 s on Apple MPS against 435 s on an RTX 4090. That ratio reflects hardware, not models.
- Ten epochs is short for this task.

What the team would try next.

- One run of Anees's architecture at a 256 character context with everything else held fixed, which isolates context length from depth.
- A longer run at the better of the two configurations.
- Measuring repetition at matched perplexity rather than matched temperature.

II. TASK 2: YELP POLARITY SENTIMENT CLASSIFICATION

A. Objective

Train three sentiment classifiers per member on Yelp Polarity, with no pretrained embeddings and no pretrained language model, report the full metric list per model, and manually review twenty of each member's own errors in four fixed categories.

B. Member 1: Anees Saheba Guddi

1) Models

TABLE IX

	Baseline	Experiment A	Experiment B
Name	baseline_max_pool	experiment_lstm	experiment_attention_pool
Encoder	masked max pooling	two layer unidirectional LSTM	learned scalar attention pooling
Embedding dimension	96	160	128
Hidden dimension	128	128	128
Attention dimension	not applicable	not applicable	64
Dropout	0.10	0.30	0.20
Parameters	2,412,545	4,297,217	3,224,962

- All three share the same vocabulary, preprocessing, classifier head and training loop, so any difference comes from the encoder alone.
- All three mask padding. The notebook asserts that adding padding does not change the baseline prediction, and the measured difference is exactly 0.0.

2) Preprocessing and hyperparameters

- **Seed 5330.**
- Lowercase, regex word split, stopwords removed, **except negation words**: not, no, never, nor, none, cannot and the contractions. Removing them makes “not good” and “good” identical.
- Vocabulary 25,000, minimum frequency 3, covering 98.29 percent of training tokens out of 198,238 distinct words.
- Maximum length 320 tokens, which keeps 92.65 percent of reviews whole.
- Batch size 256, 8 epochs, learning rate $2e-4$, weight decay $5e-4$, gradient clip 1.0.
- Split: 504,000 train, 56,000 validation, 38,000 test, all balanced 50 percent positive.

3) Results

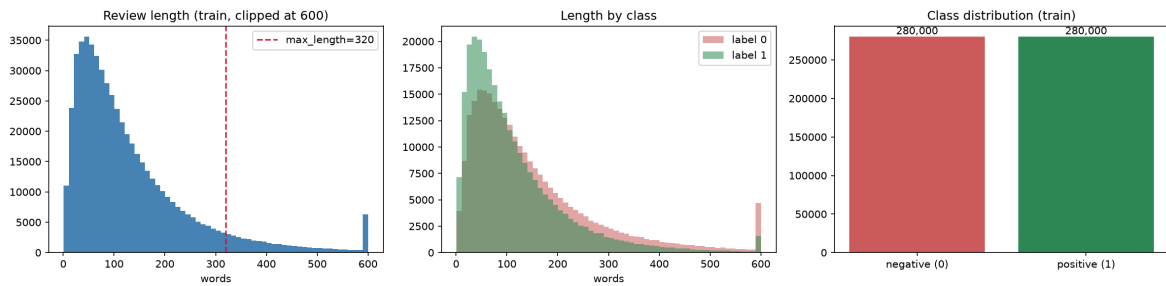


Fig. 3. Class balance and review length distribution for the Yelp Polarity split used by Anees. Both classes are exactly balanced and the 320 token limit keeps 92.65 percent of reviews whole.

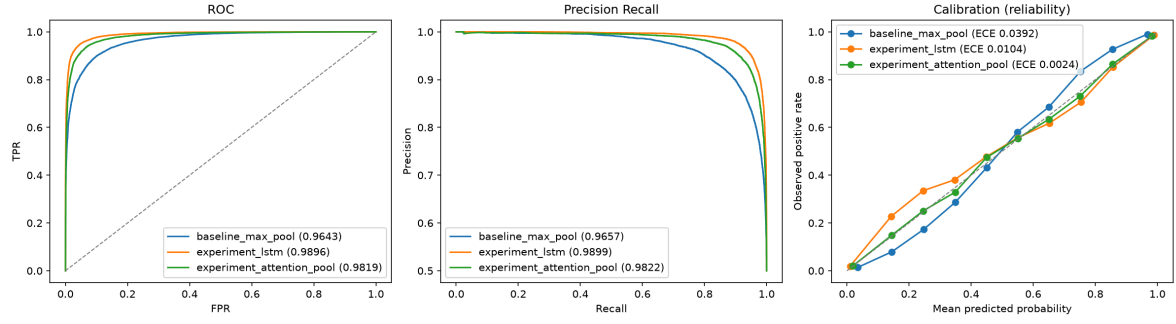


Fig. 4. ROC, precision recall and calibration curves for all three of Anees’s classifiers. Attention pooling sits closest to the diagonal on the calibration panel, which is its expected calibration error of 0.0024.

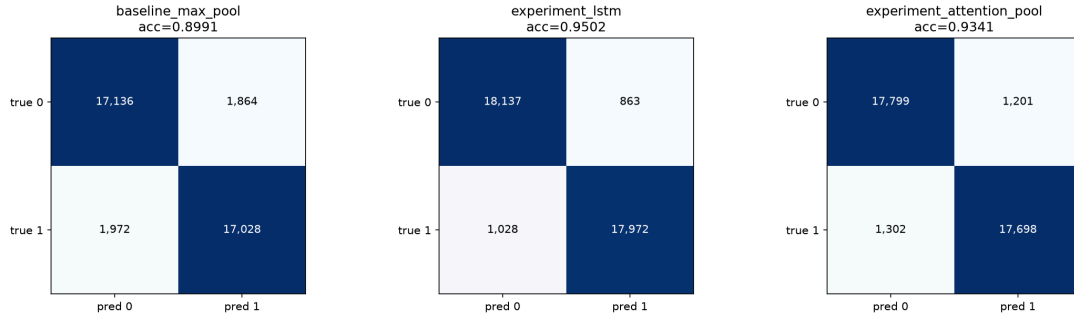


Fig. 5. Confusion matrices for the three classifiers, Anees. The LSTM reduces both false positives and false negatives relative to the max pooling baseline.

TABLE X

Metric	baseline_max_pool	experiment_lstm	experiment_attention_pool
Accuracy	0.8991	0.9502	0.9341
Macro F1	0.8991	0.9502	0.9341
ROC AUC	0.9643	0.9896	0.9819
PR AUC	0.9657	0.9899	0.9822
MCC	0.7981	0.9005	0.8683
Brier score	0.0756	0.0375	0.0490
Expected calibration error	0.0392	0.0104	0.0024
Parameters	2,412,545	4,297,217	3,224,962
Best epoch	7 of 8	6 of 8	8 of 8
Training time	408 s	9,685 s	437 s
Throughput	9,876 ex/s	416 ex/s	9,227 ex/s
Peak memory	1.048 GB	1.258 GB	1.267 GB

95 percent bootstrap confidence intervals on accuracy: baseline 0.8962 to 0.9020, LSTM 0.9481 to 0.9525, attention 0.9318 to 0.9367. None overlap.

McNemar against his own baseline:

TABLE XI

Model	Statistic	p value	Baseline right, model wrong	Model right, baseline wrong
experiment_lstm	1087.52	1.1e-252	765	2,710
experiment_attention_pool	599.20	4.1e-137	814	2,147

Error rate by slice, lower is better:

TABLE XII

Model	Short (12,574)	Medium (12,611)	Long (12,815)	Truncated (2,730)
baseline_max_pool	0.0930	0.1014	0.1082	0.0993
experiment_lstm	0.0511	0.0473	0.0508	0.0476
experiment_attention_pool	0.0687	0.0634	0.0655	0.0579

4) Error review, twenty errors

Reviewed model: experiment_lstm, 1,891 errors in 38,000 test reviews.

TABLE XIII

Error type	Count of 20
Sentiment reversal, verdict contradicts the bulk	4
Concession or contrast structure	4
Mixed or hedged sentiment	4
Third party or narrative sentiment	2
Label noise, the gold label looks wrong	2
Vocabulary gap, rare or informal words	2
Truncation beyond 320 tokens	2

Representative cases:

- **Confident false positive, p 0.9988.** “NOTE: This was a 4 star review, but the food quality and ESPECIALLY customer service have gone down the tubes” followed by 400 words of praise. The verdict is in the first sentence and the rest contradicts it.
- **Confident false negative, p 0.0009.** “This place is so much better since they changed owners ... it was terrible ... Now its much better.” Temporal reversal, where the negative past takes more words than the positive present.
- **Near threshold, p 0.4986.** “the staff are really knowledgeable ... The store area itself can feel kind of cramped, but I’d much rather spend less time getting the right thing.” A complaint acknowledged then dismissed, which the model reads as a complaint.

His findings.

- Eight of twenty errors turn on a reversal or a concession, by far the largest group, making contrast marker weighting the first fix to try.
- Two of twenty confident errors look like wrong gold labels. If that rate holds across all 1,891 errors, roughly 190 are not the model’s fault.
- Truncation is **not** the problem it appears to be. The truncated slice has a lower error rate than the long slice for all three of his models, so raising the length limit is the obvious fix and the wrong one.

C. Member 2: Yashashree Shinde

1) Models

TABLE XIV

	Baseline	Experiment A	Experiment B
Name	baseline_mean_pool	experiment_cnn	experiment_bigru
Encoder	masked mean pooling	multi width convolutions	bidirectional GRU
Parameters	3,856,770	5,181,890	6,057,026

- The baseline averages evidence across every token, since sentiment is usually spread across a review rather than concentrated in one word.
- The convolutional model uses several kernel widths to capture local phrases such as “not good” as single features.
- The bidirectional GRU reads the review in both directions, so it can represent word order and carry negation scope across a span.

2) Preprocessing and hyperparameters

- **Seed 4349.**
- Lowercase, word split, stopwords removed except negation words.
- Vocabulary 30,000, minimum frequency 2, deliberately larger and more permissive than her teammate’s so that food and service specific words survive.
- Maximum length 256 tokens.
- Batch size 256, 5 epochs, learning rate 3e-4, weight decay 1e-4, gradient clip 1.0.
- Validation fraction 0.10, bootstrap samples 1,000.

3) Results

TABLE XV

Metric	baseline_mean_pool	experiment_cnn	experiment_bigr
Accuracy	0.9348	0.9285	0.9469
Macro F1	0.9348	0.9285	0.9469
ROC AUC	0.9822	0.9818	0.9869
PR AUC	0.9824	0.9823	0.9871
MCC	0.8696	0.8579	0.8938
Brier score	0.0482	0.0533	0.0412
Expected calibration error	see note	see note	see note
Parameters	3,856,770	5,181,890	6,057,026
Training time	256 s	293 s	290 s
Peak memory	0.16 GB	0.45 GB	1.80 GB

95 percent bootstrap confidence intervals on accuracy: baseline 0.9324 to 0.9373, CNN 0.9263 to 0.9308, BiGRU 0.9447 to 0.9492. The BiGRU interval overlaps neither of the others.

Note on calibration. The reported expected calibration errors of 0.4364, 0.4525 and 0.4629 cannot be correct alongside the Brier scores of 0.0482, 0.0533 and 0.0412.

- For a balanced test set the Brier decomposition gives Brier at least equal to reliability, and reliability is at least the square of the expected calibration error.
- An error of 0.4629 therefore requires a Brier of at least 0.214, against a reported 0.0412.
- All three models break that bound, so the calculation is wrong rather than the models being badly calibrated.
- The Brier scores themselves are sound and comparable to her teammate’s, which is the better evidence that her probabilities are fine.
- The values remain in her metrics file for transparency but are not quoted in any comparison.

McNemar against her own baseline:

TABLE XVI

Model	Chi square	p value	Baseline right, model wrong	Model right, baseline wrong
experiment_bigr	109.88	1.0e-25	725	1,184
experiment_cnn	22.74	1.9e-06	1,376	1,136

The convolutional result is significant but the counts run against it. The baseline is right and the CNN wrong on 1,376 reviews, against 1,136 the other way, so the CNN is significantly **worse** than its own baseline.

Error rate by slice:

TABLE XVII

Model	Short (12,899)	Medium (14,843)	Long (10,258)	Contains negation (23,262)
baseline_mean_pool	0.0648	0.0632	0.0685	0.0674
experiment_cnn	0.0723	0.0689	0.0743	0.0704
experiment_bigr	0.0549	0.0492	0.0565	0.0532

4) Error review, twenty errors

Reviewed model: experiment_bigr, about 2,019 errors in 38,000 test reviews.

TABLE XVIII

Error type	Count of 20
Negation over a long span	5
Mixed or aspect split sentiment	5
Truncation at 256 tokens	4
Sarcasm or irony	3
Temporal or structural reversal	2
Too little signal	1

Representative cases:

- **Confident false positive, p 0.9998.** “rib eye tasty little over cooked didn’t complain because flavor still fantastic ... i’d recommend ruth chris lg’s flemings morton’s”. Damning with faint praise, recommending four competitors instead.
- **Confident false negative, p 0.0004.** “wonderful market extensive variety products rated market 4 only rate customer service 1”. A split rating across two aspects in one sentence.
- **Near threshold, p 0.4994.** “can’t complain about food everything pretty good”. A positive idiom built from negative words.
- **Slice failure, p 0.9022, 256 tokens.** “mirrored ceilings velvet upholstered chairs cheetah print fabric like 80’s movie scarface”. Sustained sarcasm, where descriptive words read as positive and the tone is mocking.

Her findings, including one she corrected.

- Her first reading was that negation was the main failure mode, because 16 of the 20 sampled errors sit in a slice containing negation.
- Her own slice metrics refute it. Reviews containing negation fail at 0.0532 against 0.0531 for the whole test set, so there is no effect at all.
- The explanation is the base rate: 61.2 percent of all test reviews contain negation, so about 12 of 20 would be expected anyway. A binomial test on 16 of 20 against that base rate gives $p = 0.063$.

- A hand inspection of twenty errors is a good way to generate a hypothesis and a bad way to test one, because an error sample carries no information about the base rate it was drawn from.
- What the slice metrics do support: long reviews are her worst slice at 0.0565 against 0.0492 for medium, and all five of her sampled long review failures sit at exactly her 256 token limit.

D. Side by Side Comparison

	Anees baseline	Anees exp A	Anees exp B	Yash baseline	Yash exp A	Yash exp B
Encoder	max pooling	2 layer LSTM	attention pooling	mean pooling	multi CNN	width bidirectional GRU
Vocabulary	25,000	25,000	25,000	30,000	30,000	30,000
Minimum frequency	3	3	3	2	2	2
Maximum length	320	320	320	256	256	256
Learning rate	2e-4	2e-4	2e-4	3e-4	3e-4	3e-4
Epochs	8	8	8	5	5	5
Seed	5330	5330	5330	4349	4349	4349
Parameters	2,412,545	4,297,217	3,224,962	3,856,770	5,181,890	6,057,026
Accuracy	0.8991	0.9502	0.9341	0.9348	0.9285	0.9469
Macro F1	0.8991	0.9502	0.9341	0.9348	0.9285	0.9469
ROC AUC	0.9643	0.9896	0.9819	0.9822	0.9818	0.9869
PR AUC	0.9657	0.9899	0.9822	0.9824	0.9823	0.9871
MCC	0.7981	0.9005	0.8683	0.8696	0.8579	0.8938
Brier	0.0756	0.0375	0.0490	0.0482	0.0533	0.0412
Calibration error	0.0392	0.0104	0.0024	invalid	invalid	invalid
Training time	408 s	9,685 s	437 s	256 s	293 s	290 s
Peak memory	1.05 GB	1.26 GB	1.27 GB	0.16 GB	0.45 GB	1.80 GB
Hardware	M4 MPS	M4 MPS	M4 MPS	RTX 4090	RTX 4090	RTX 4090

E. Joint Analysis

Strengths.

- Both members' best models are recurrent, and both beat their own baselines by margins that are not close to chance.
- The best model across the team is Anees's LSTM at 95.02 percent, with Yashashree's bidirectional GRU at 94.69 percent and 29 percent more parameters.
- That both members independently arrived at a recurrent model as strongest is the clearest shared result: on this dataset, reading word order beats every order free method either of them tried.
- Attention pooling is the best calibrated model either member produced, with an expected calibration error of 0.0024, sixteen times better than the max pooling baseline.

Weaknesses.

- Yashashree's convolutional model is significantly worse than her own baseline despite 34 percent more parameters. A convolution reaches only as far as its widest kernel, so a negation separated from its target by several words is out of range, while mean pooling at least accumulates every token evenly.
- Anees's max pooling baseline is the weakest model in the team at 89.91 percent.
- Anees's attention pooling was still improving at its final epoch, so its 93.41 percent is a lower bound rather than a ceiling.
- Anees's LSTM began overfitting from epoch 7 and was saved only by selecting on validation loss.

The most useful disagreement is between the two baselines.

- Mean pooling reaches 93.48 percent where max pooling reaches 89.91, a gap of 3.6 points between two models that differ only in how they collapse positions into one vector.
- Max pooling takes the single most extreme activation per dimension, so one strong word can decide a review. Mean pooling accumulates evidence across every token.
- For sentiment this matters, because a long review usually carries many mild signals rather than one decisive word.
- Anees's error review supports this from the other direction: eight of his twenty errors turn on a contrast or a concession, exactly where one extreme token overrides the balance of the text.
- Had the team known this before running, mean pooling would have been the better baseline for both members.

The two error reviews appear to disagree about truncation, and the disagreement resolves.

- Anees truncates at 320 tokens and sees no truncation effect. His truncated slice has a lower error rate than his long slice for all three models.
- Yashashree truncates at 256 and sees a clear one. All five of her long review failures sit at exactly that limit, and long reviews are her worst slice.
- These are consistent rather than contradictory. The cost of truncation appears somewhere between 256 and 320 tokens on this dataset.

Limitations.

- Training times are not comparable across Apple MPS and an RTX 4090.
- Anees ran 8 epochs and Yashashree 5, so the comparison is not purely architectural.
- Yashashree's calibration numbers are invalid and cannot enter the comparison.
- Anees's LSTM is not bit reproducible on MPS. Across two runs its best epoch moved from 5 to 6, while his other two models reproduced exactly, so the cause is the LSTM kernels rather than the seeding.

What the team would try next.

- Mean pooling as the baseline for both members.
- A length ablation at 512 tokens, which Yashashree's slice metrics motivate.
- Contrast marker weighting, which Anees's error review motivates.
- Recomputing Yashashree's calibration, most likely taking confidence as the maximum of p and $1 - p$ rather than p of the positive class.

III. TASK 3: CYCLEGAN MONET AND PHOTO STYLE TRANSFER

A. Objective

Train a CycleGAN from scratch to translate in both directions between Monet paintings and photographs, report the full metric list in both directions, verify cycle consistency, analyse visual failure cases, and submit to the Kaggle class competition.

The data is unpaired, so no photograph is the same scene as any painting and there is no target image to compare an output against.

B. Member 1: Anees Saheba Guddi

1) Architecture

TABLE XIX

Component	Value
Generator	resnet, 6 residual blocks
Base filters	80
Downsampling	two stride 2 convolutions
Upsampling	two transposed convolutions
Normalisation	instance
Padding	reflection
Discriminator	PatchGAN, 16 by 16 output grid
Adversarial loss	least squares
Parameters per generator	12,239,363
Parameters per discriminator	2,764,737
Total parameters	30,008,200

Reasons for the choices:

- **Six blocks on 80 filters** rather than the standard nine on 64. This spends almost the same parameter budget on a wider and shallower bottleneck. A wider bottleneck carries more feature channels at the smallest spatial size, which is where the texture and colour palette of a style are represented.
- **Instance normalisation**, which normalises per image rather than per batch and suits style transfer where each image has its own colour statistics. It is also necessary at batch size 1.
- **Reflection padding**, which avoids the dark border that zero padding leaves at image edges.
- **PatchGAN** judges local patches rather than the whole image, which forces local texture to look painterly instead of only the global colour being right.

2) Hyperparameters

TABLE XX

Hyperparameter	Value
Seed	5330
Image size	256
Batch size	1
Epochs	50
Learning rate	2e-4
Betas	0.5 and 0.999
Schedule	constant to epoch 25, then linear decay to zero
Lambda cycle	10.0
Lambda identity	2.5
Gradient clip	1.0

- **Image size 256** because it is the size the competition scorer uses and the size his teammate trained at, so the two runs are comparable.
- **Lambda identity 2.5** rather than the usual 5.0. Identity loss asks the generator to leave an image alone when it is already in the target domain, which mostly holds the colour palette fixed. The Monet direction is scored on how repainted the output looks, so a strong identity term works against the measurement. Lowering it frees the generator to repaint.

3) Results

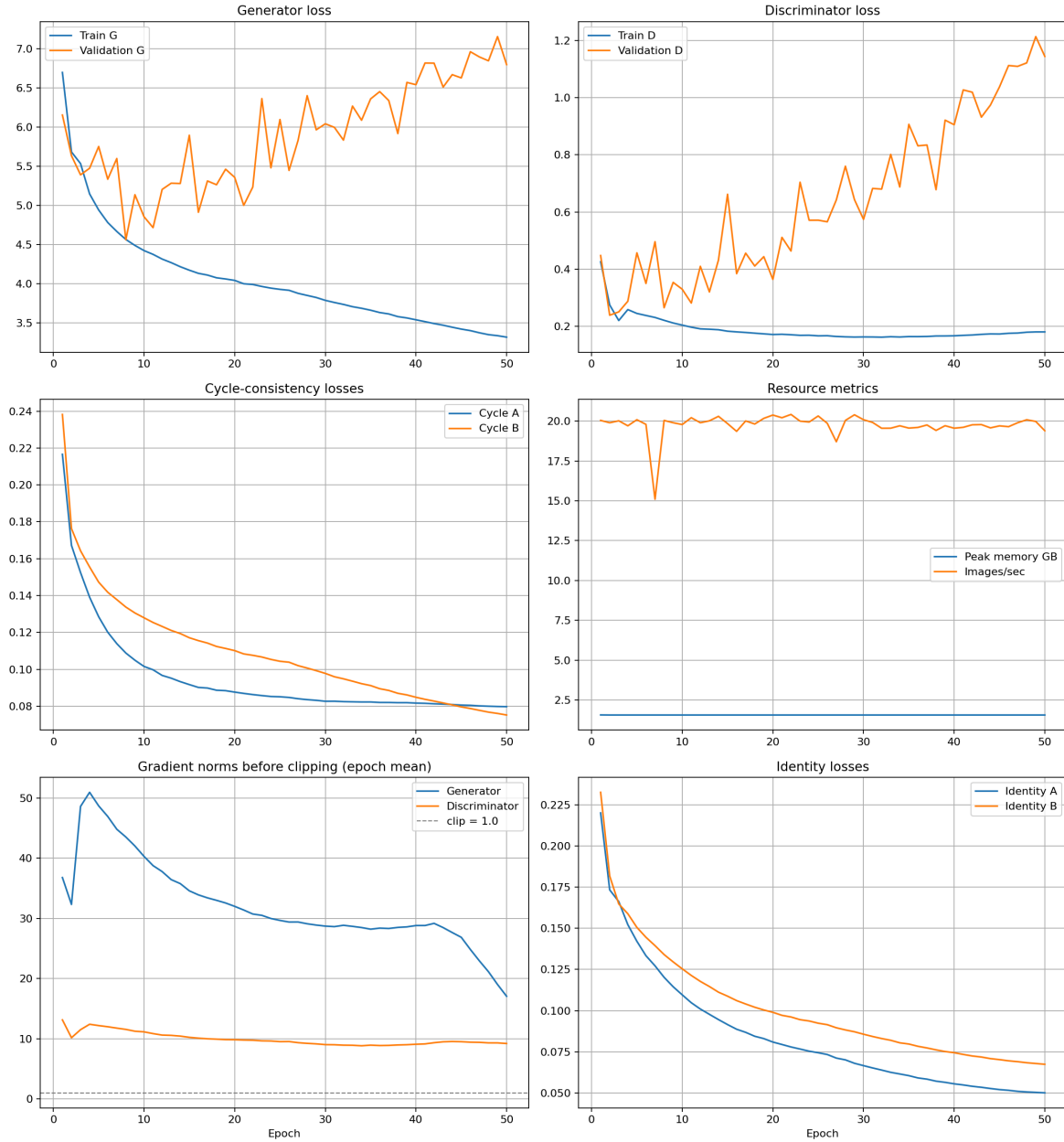


Fig. 6. CycleGAN training curves over 50 epochs, Anees. Generator and discriminator losses, cycle losses, identity losses and gradient norms. No loss spikes and no non finite steps.

TABLE XXI

Metric	Monet to photo	Photo to Monet
FID	178.94	86.59
KID	0.0328	0.0075
KID standard deviation	0.0050	0.0004
Density	0.9333	0.5420
Coverage	0.0117	0.7267
Cycle reconstruction L1	0.1647	0.0949
Content cosine similarity	0.7348	0.8658
LPIPS	0.3556	0.3820

TABLE XXII

Cost	Value
Parameters	30,008,200
Training time	32,198 s, 8.94 hours
Throughput	19.76 images per second
Peak memory	1.568 GB
Non finite steps	0
Optimiser steps	316,750
GPU	NVIDIA RTX 4090

Training stability:

Epoch	Generator	Discriminator	Cycle A	Cycle B	Generator gradient norm
1	6.6975	0.4254	0.2166	0.2382	36.78
11	4.3762	0.1966	0.0997	0.1254	38.77
21	4.0013	0.1718	0.0869	0.1083	31.39
31	3.7625	0.1623	0.0827	0.0960	28.63
41	3.5178	0.1681	0.0815	0.0837	28.81
50	3.3199	0.1800	0.0797	0.0752	17.05

Kaggle submission:

- Public leaderboard score **-52.8390**
- Submitted FID 105.2606, MiFID 0.4175
- Per direction under the scorer: photo to Monet FID 95.4884, Monet to photo FID 115.0328, each on 300 pairs

4) Failure analysis

Case 1, hallucinated signature. File validation_B2A/0000.png.

- A cursive artist signature appears in the lower right corner of a translated lighthouse photograph. Nothing resembling text exists in the input.
- Monet paintings are signed, usually in a lower corner, so the discriminator learns that a small region of dark cursive strokes there is evidence of a real Monet, and the generator produces one because it lowers the adversarial loss.
- Nothing in the objective distinguishes style from provenance marks.
- Measured across all 30 validation translations by comparing gradient energy in the lower right against the rest of each image, 7 of 30 have a ratio above 1.25, with a maximum of 1.97.
- This matters for the submission, because a painted signature is exactly the kind of local texture that improves FID while being obviously wrong to a person.

Case 2, repeated texture blobs and colour banding. File validation_B2A/0007.png.

- A near featureless sky comes back filled with repeated flower shaped blobs on a rough grid, with colour banding through green, pink and blue.
- The track and ground in the same image, which have real structure, translate cleanly.
- The grid regularity is the signature of transposed convolution. The upsampling layers use kernel size 3 with stride 2, and because 3 does not divide evenly by 2 some output pixels receive more contributions than others at a fixed spacing.

- The blob content is a second cause. A flat region gives the generator almost no signal to preserve while the discriminator still demands painterly texture, so the generator invents brushwork.
- The fix to test is nearest neighbour upsampling followed by a plain convolution.

Case 3, colour shift in cycle reconstruction. Monet haystacks, file validation_cycle_A/0000.png.

- The input is Monet's haystacks in warm pink and orange light. After going to the photo domain and back, every shape returns in the right place but the palette has moved to cool blue and cyan.
- This is the measured cycle reconstruction L1 of 0.1647 seen as an image: structure recovered, colour not.
- It is the visible cost of running identity loss at 2.5. His teammate's figure on the same direction is 0.1411.

Training stability observations.

- Zero non finite steps across 316,750 optimiser steps.
- No loss spikes and no sign of the discriminator winning. Discriminator loss settles near 0.17 and stays there.
- Generator gradient norms fall steadily from 37 to 17. The sharp drop over the final ten epochs follows the learning rate decaying to zero.
- Validation generator loss **rises** while training loss falls, from 6.15 to 6.80 against 6.70 down to 3.32. In a supervised model that reads as overfitting. Here it means less, because the validation generator loss is measured against a discriminator that is itself still improving, so the generator's loss can rise simply because its opponent got better. The numbers that track quality are the cycle losses, which fall throughout, and FID and KID from the final checkpoint.

C. Member 2: Yashashree Shinde

1) Architecture

TABLE XXIII

Component	Value
Generator	resnet, 9 residual blocks
Base filters	64
Downsampling	two stride 2 convolutions
Upsampling	two transposed convolutions
Normalisation	instance
Padding	reflection
Discriminator	PatchGAN, 16 by 16 output grid
Adversarial loss	least squares
Parameters per generator	11,378,179
Total parameters	28,285,832

Reasons for the choices:

- **Nine residual blocks on 64 base filters** is the standard CycleGAN generator for 256 by 256 inputs and the configuration the paper uses for the Monet task. She wanted the reference architecture as her starting point.
- The remaining components match the reference implementation.

2) Hyperparameters

TABLE XXIV

Hyperparameter	Value
Seed	4349
Image size	256
Batch size	1
Epochs	50
Learning rate	2e-4
Betas	0.5 and 0.999
Schedule	constant to epoch 25, then linear decay to zero
Lambda cycle	10.0
Lambda identity	5.0
Gradient clip	1.0

- **Lambda identity 5.0**, half the cycle weight, which is the usual setting. Identity loss holds the colour palette steady, which is why the CycleGAN paper adds it for the painting task.

3) Results

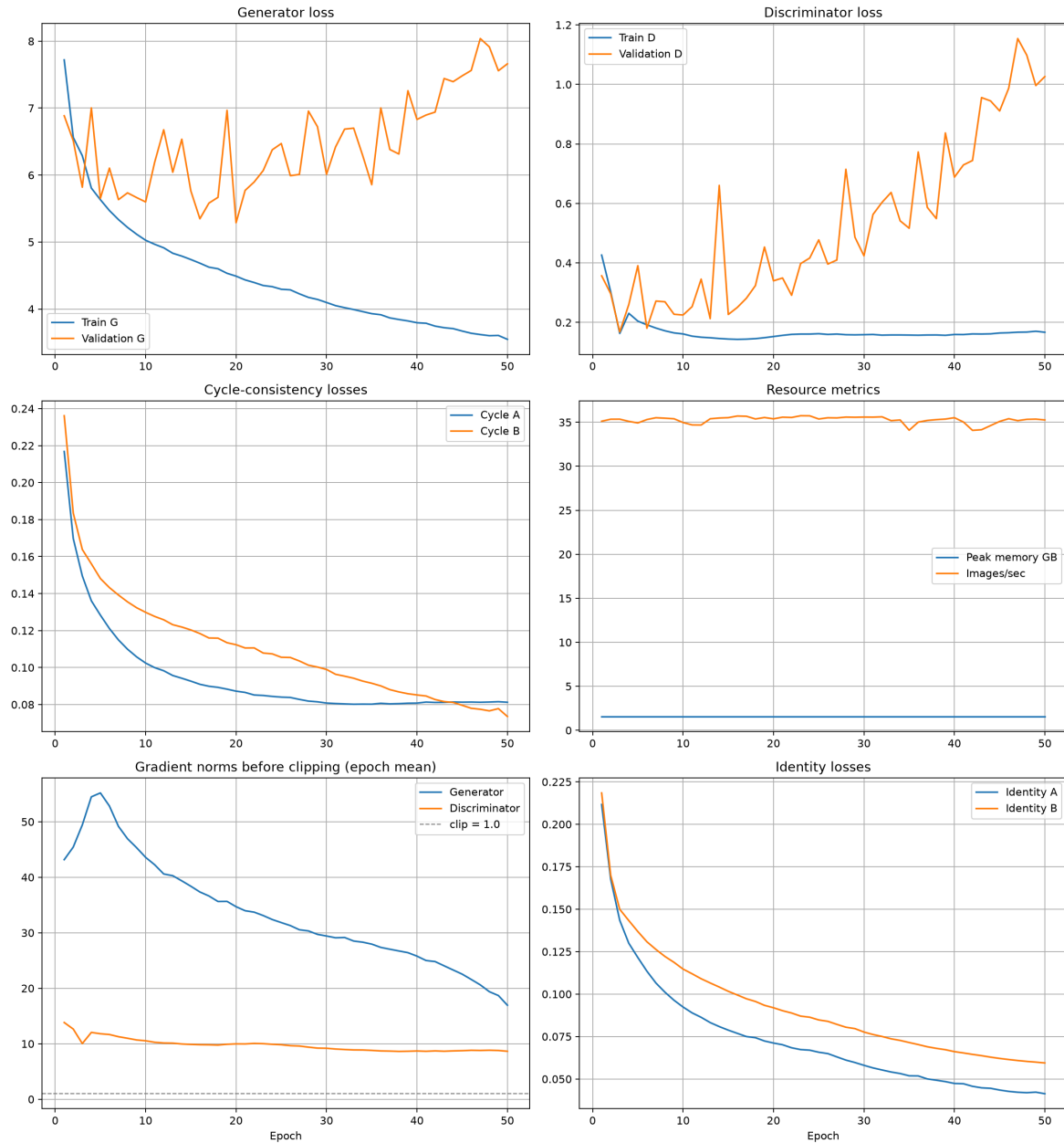


Fig. 7. CycleGAN training curves over 50 epochs, Yashashree. The same stability pattern, with gradient norms falling from 43 to 17.

TABLE XXV

Metric	Monet to photo	Photo to Monet
FID	189.22	92.39
KID	0.0431	0.0122
KID standard deviation	0.0057	0.0006
Density	0.6222	0.4163
Coverage	0.0070	0.6867
Cycle reconstruction L1	0.1411	0.0962
Content cosine similarity	0.8011	0.7463
LPIPS	0.3620	0.4177

TABLE XXVI

Cost	Value
Parameters	28,285,832
Training time	18,051 s, 5.01 hours
Throughput	35.26 images per second
Peak memory	1.563 GB
Non finite steps	0
GPU	NVIDIA RTX 5090

Training stability:

Epoch	Generator	Discriminator	Cycle A	Cycle B	Generator gradient norm
1	7.7211	0.4264	0.2168	0.2362	43.17
10	5.0264	0.1614	0.1024	0.1299	43.59
20	4.4901	0.1527	0.0872	0.1123	34.70
30	4.0969	0.1589	0.0808	0.0990	29.42
40	3.7943	0.1596	0.0808	0.0852	25.79
50	3.5460	0.1671	0.0812	0.0735	16.94

Kaggle submission:

- Public leaderboard score **-54.6990**
- Submitted FID 108.9811, MiFID 0.4171

4) Failure analysis

Case 1, under stylisation. File validation_B2A/0000.png.

- The translated lighthouse photograph keeps its structure, its lighting and most of its colour. Brush texture has been laid over the surface but the scene has not been repainted.
- The sky gradient, the red railing and the water all sit where the camera put them, in the shades the camera recorded.
- The cause is the identity weight of 5.0, which penalises changing an image already in the target domain and so holds the colour palette.
- The distribution metrics reward the opposite, since a Monet is defined partly by a palette that departs from the photographic one.
- Her teammate ran identity weight 2.5 and reached FID 86.59 against her 92.39. The visual difference on this same input is exactly this: his is repainted, hers is a photograph with texture.

Case 2, checkerboard colour blocks. File validation_B2A/0007.png.

- A bright block of saturated blue and red appears on the left of a snow covered mountain scene, roughly 30 pixels across and arranged on a visible grid. Nothing in the input corresponds to it.
- The surrounding terrain translates cleanly.
- The cause is the same transposed convolution imbalance described in her teammate's case 2, kernel size 3 against stride 2.
- It appears in flat regions because there is no strong input signal to dominate the uneven overlap.

Case 3, cycle reconstruction, included as a counterexample. Input monet_jpg/bc4b364a44.jpg, reconstruction validation_cycle_A/0000.png.

- A Monet of a valley with poplars, cottages and a wooden fence is translated to a photograph and back.

- The reconstruction returns the composition and the palette, the blue sky, the ochre hill, the green field. The visible loss is in fine detail where brush strokes have been softened.
- This is the measured cycle reconstruction L1 of 0.1411 shown as an image, and it is the better of the two runs on this metric.
- It is included because it is the direct consequence of the choice that caused case 1. The identity weight that holds her outputs too close to the source is the same term that makes her round trip faithful.

Measured artifact rate.

TABLE XXVII

	Mean ratio	Median	Maximum	Above 1.25
Yashashree	1.27	1.26	2.73	16 of 30
Anees	1.02	0.99	1.55	7 of 30

- More than half of her validation outputs carry elevated structure along the bottom edge, against roughly a quarter of her teammate's.
- Part of this is the painted signature both models learned. The larger ratio suggests the deeper nine block generator reproduces that artifact more strongly.

Training stability observations.

- Zero non finite steps across all 50 epochs.
- No loss spikes and no sign of the discriminator winning. Discriminator loss settles near 0.16.
- Generator gradient norms fall from 43 to 17.
- Validation generator loss rises from 6.89 to 7.66 while training loss falls from 7.72 to 3.55, for the same reason given in her teammate's section.

D. Human Audit

The brief asks for a blinded audit of 30 fixed samples on style, content and artifacts, scored by two raters, with inter rater agreement reported.

Status: not completed. Both `outputs/metrics/human_audit_30_samples.csv` files are 30 row templates with empty rater columns. The notebook deliberately does not generate ratings, and no substitute for two human raters was used. The automated preparation is in the repository and is described below, so the audit can be completed without rerunning anything.

1) What was prepared

- Contact sheets in `task3_gan/audit_sheets/` place all 30 samples per member per direction on a single labelled image, so a rater views one picture rather than opening thirty files.
- The audit files originally pointed at `/app/task3_gan_output/...`, paths inside the GPU lab container that do not exist in the repository. They are now repository relative.
- `scripts/score_human_audit.py` reads whatever two raters enter and computes per criterion means, exact agreement, agreement within one point, and Cohen's kappa with quadratic weights. Quadratic weighting is the appropriate statistic for an ordinal five point scale because it corrects for chance agreement and penalises a two point disagreement four times as heavily as a one point one. The implementation was checked against identical, one point apart, random and realistic rating pairs, returning 1.000, 0.000, 0.143 and 0.898 respectively.

2) Why this component matters more than its mark weight

Every automatic metric in this task is computed from Inception features or pixel distances. None of them penalise a plausible but fabricated detail. The clearest example is in both members' outputs.

Both generators paint imitation artist signatures into the lower part of the frame, because real Monet paintings are signed and the discriminator therefore treats a corner mark as evidence of authenticity. Measured by comparing gradient energy in the bottom strip of each image against the rest of it, the artifact appears in 7 of 30 of Anees’s validation translations and 16 of 30 of Yashashree’s, with ratios up to 1.55 and 2.73.

A signature of this kind very likely *improves* FID, since it moves the output closer to the real Monet distribution in feature space. It is also obviously wrong to any person who looks at the image. Both members’ Kaggle submissions contain it. This is exactly the class of defect a human audit exists to catch, and the reason its absence is a real gap rather than a missing formality.

3) *What the audit would have been expected to show*

The automatic metrics give a clear prediction, which is what makes the missing audit measurable in principle rather than merely absent.

TABLE XXVIII

Criterion	Expected ordering	Supporting metric
Style	Anees ahead	photo to Monet FID 86.59 against 92.39
Content	Yashashree ahead	cycle reconstruction L1 0.1411 against 0.1647
Artifacts	Anees ahead	signature artifact in 7 of 30 against 16 of 30

An audit that reproduced this ordering would corroborate the automatic metrics. An audit that contradicted it would be the more interesting result, because it would indicate that FID and KID are rewarding something human raters do not value, which the signature artifact already suggests is happening to some degree.

E. Side by Side Comparison

TABLE XXIX

	Anees	Yashashree
Residual blocks	6	9
Base filters	80	64
Parameters per generator	12,239,363	11,378,179
Total parameters	30,008,200	28,285,832
Discriminator	PatchGAN 16 by 16	PatchGAN 16 by 16
Normalisation	instance	instance
Adversarial loss	least squares	least squares
Image size	256	256
Batch size	1	1
Epochs	50	50
Learning rate	2e-4	2e-4
Lambda cycle	10.0	10.0
Lambda identity	2.5	5.0
Seed	5330	4349
FID, Monet to photo	178.94	189.22
FID, photo to Monet	86.59	92.39
KID, Monet to photo	0.0328	0.0431
KID, photo to Monet	0.0075	0.0122
Density, Monet to photo	0.9333	0.6222
Density, photo to Monet	0.5420	0.4163
Coverage, Monet to photo	0.0117	0.0070
Coverage, photo to Monet	0.7267	0.6867
Cycle L1, Monet side	0.1647	0.1411
Cycle L1, photo side	0.0949	0.0962
Content cosine, Monet to photo	0.7348	0.8011
Content cosine, photo to Monet	0.8658	0.7463
LPIPS, Monet to photo	0.3556	0.3620
LPIPS, photo to Monet	0.3820	0.4177
Non finite steps	0	0
Training time	32,198 s	18,051 s
Throughput	19.76 img/s	35.26 img/s
Peak memory	1.568 GB	1.563 GB
Hardware	RTX 4090	RTX 5090
Kaggle public score	-52.8390	-54.6990

On the Kaggle metric. The competition takes a one row submission file with ID, FID and MiFID, scored by the instructor’s evaluation script, and the leaderboard shows the negative mean of the two. A less negative score is better. The FID values in this table differ from the submitted ones because they are not the same measurement: each member’s notebook compares 300 real Monets against all generated images with its own Inception preprocessing, while the scorer subsamples both sides to 300, sorts by filename and resizes differently. Only the scorer’s version is comparable across the class.

F. Joint Analysis

Strengths.

- Both runs completed 50 epochs with zero non finite steps, no loss spikes and no sign of either discriminator overpowering its generator.
- Both discriminator losses settled near 0.16 to 0.17 and stayed there, which is roughly where they should sit when neither network dominates.
- Both models translate photographs into recognisably Monet like images.
- Both reconstruct the round trip closely enough to confirm cycle consistency is holding, which is the assumption the whole method rests on.
- Both members submitted successfully to the Kaggle competition.



Fig. 8. The same input photograph translated to Monet by each model. Left, Anees at identity weight 2.5, repainted. Right, Yashashree at identity weight 5.0, still reading as a photograph with brush texture applied.



Fig. 9. Cycle reconstruction, Anees. Left, the original Monet. Right, the reconstruction after a round trip through the photo domain. Structure returns, the palette shifts from warm to cool, which is the measured L1 of 0.1647.



Fig. 10. Cycle reconstruction, Yashashree. Left, the original Monet. Right, the reconstruction. Both structure and palette return, which is the measured L1 of 0.1411.

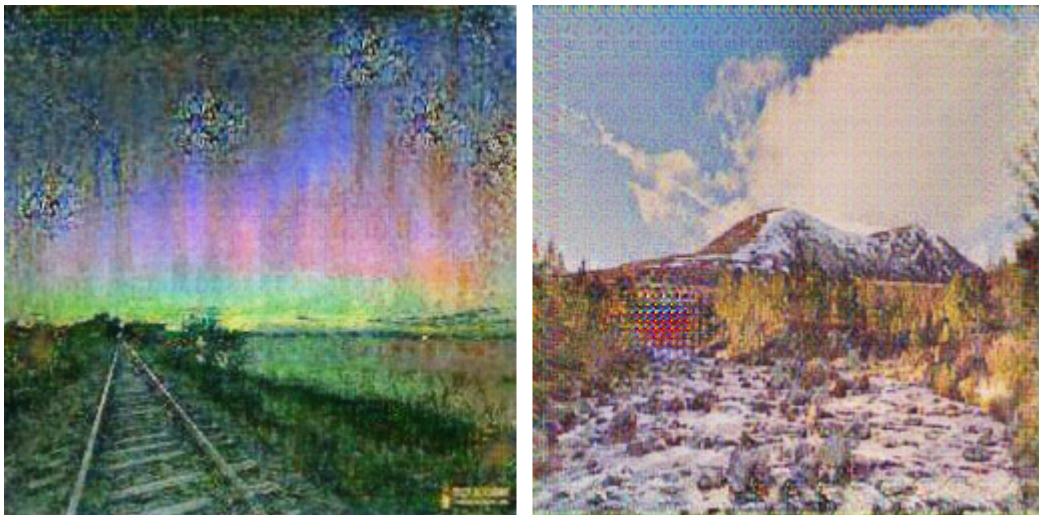


Fig. 11. Artifacts in flat regions. Left, Anees, repeated blob texture and colour banding across the sky. Right, Yashashree, a saturated checkerboard block on the left of the frame. Both come from the transposed convolution kernel and stride mismatch.

The comparison separates cleanly along one hyperparameter.

- The two runs used the same data, epochs, resolution, learning rate and schedule. Three things differ: the seed, the generator shape and the identity weight.
- Anees is better on every FID and KID number in both directions.
- Yashashree is better on cycle reconstruction for the Monet direction, 0.1411 against 0.1647, and on content similarity, 0.8011 against 0.7348.
- That is exactly the trade the identity term controls. At 2.5 the generator is freer to repaint, which the distribution metrics reward. At 5.0 the palette is held, which keeps the round trip faithful and leaves outputs closer to the source photograph.
- The effect is visible as well as measurable. The same lighthouse photograph is repainted by one model and comes back from the other still reading as a photograph with brush texture applied.

Weaknesses.

- Both models paint fake artist signatures into the lower part of the frame. Real Monets are signed, so a corner mark is evidence of authenticity to the discriminator. Measured over 30 validation translations each, the

artifact appears in 7 of 30 of Anees's outputs and 16 of 30 of Yashashree's.

- No automatic metric penalises this, and both members' Kaggle submissions contain it.
- Both runs show grid structured artifacts in flat regions, from the same transposed convolution kernel and stride mismatch.
- The Monet to photo direction is weak for both, FID 178.94 and 189.22.

Limitations.

- Three variables changed at once between the runs, so neither member can attribute the difference to the identity weight alone.
- The Monet to photo direction is measured on only 30 generated images against 7,038 real photographs, which depresses coverage to 0.0117 and 0.0070 and makes that figure less reliable than the reverse direction.
- Both runs used 50 epochs where the CycleGAN paper uses 200 for this task, and both members' cycle losses were still falling slowly at the end.
- Training times are not comparable across an RTX 4090 and an RTX 5090.

What the team would try next.

- A single run with only the identity weight changed, holding generator shape and seed fixed, which would turn the correlation observed here into an attribution.
- Replacing the transposed convolutions with nearest neighbour upsampling followed by a plain convolution, the standard fix for the grid artifact.
- Masking signatures out of the training paintings, to test whether the signature artifact is learned from them as the team believes.
- Generating more images in the Monet to photo direction so that its FID and coverage become reliable.

IV. EVIDENCE TRAIL

Every number in this report is traceable to a committed file in the repository.

TABLE XXX

Evidence	Location
Raw training logs, unedited	reproducibility/raw_logs/
Manifests, one per member per task	reproducibility/manifests/
Task 1 metrics	task1_llm/MEMBER/metrics_report.csv
Task 1 loss curves	task1_llm/anees_saheba/outputs/loss_curves.png and task1_llm/yashashree_shinde/outputs/task1_loss_curves.svg
Task 1 generated samples	task1_llm/MEMBER/outputs/
Task 1 checkpoints	task1_llm/anees_saheba/checkpoints/best_model.pt and task1_llm/yashashree_shinde/checkpoints/task1_best_model.pt
Task 1 notebooks	task1_llm/MEMBER/src/
Task 2 metrics	task2_sentiment/MEMBER/metrics_report.csv
Task 2 slice metrics	task2_sentiment/anees_saheba/metrics_report.csv and task2_sentiment/yashashree_shinde/task2_slice_metrics.json
Task 2 significance tests	same files, plus task2_mcnemar_results.json
Task 2 error reviews	task2_sentiment/MEMBER/ error review csv and failure_analysis.md
Task 2 checkpoints	task2_sentiment/MEMBER/checkpoints/, three per member
Task 3 metrics	task3_gan/MEMBER/metrics_report.csv and outputs/metrics/full_metrics_report.json
Task 3 training curves	task3_gan/MEMBER/outputs/figures/training_curves.png
Task 3 sample translations	task3_gan/MEMBER/outputs/validation_A2B, validation_B2A, validation_cycle_A, validation_cycle_B
Task 3 checkpoints	task3_gan/MEMBER/checkpoints/generators_epoch_050.pt
Kaggle submissions	task3_gan/MEMBER/outputs/submission.csv
One command reproduction	scripts/smoke_test.sh

Notes on the evidence:

- Checkpoints are tracked with Git LFS.
- Optimiser state was removed from Yashashree’s Task 1 and Task 2 checkpoints before committing, reducing them from 206 MB to 70 MB. Adam moment buffers are needed only to resume training, not to load a model and reproduce metrics. Parameter counts were verified against the reported values after stripping.
- Anees’s Task 1 raw logs include a partial run stopped by an external time limit and a superseded run at a different seed. Both are kept unedited and renamed to mark their status, because section 5 asks for logs to be preserved rather than removed.
- Anees’s Task 2 raw logs include a failed run. Attention pooling returned NaN on reviews that stopword removal had emptied, because masking every position and taking a softmax is zero divided by zero. The fix and a one second check that catches it are in the notebook, and the failed log is kept as evidence.

V. KNOWN GAPS

Stated rather than hidden.

- **The human audit is not complete for either member.** Both human_audit_30_samples.csv files are 30 row templates with empty rater columns. The notebooks deliberately do not generate ratings. This is the only part of the Task 3 evaluation that would catch the signature artifact, since no automatic metric penalises it.
- **Yashashree’s Task 1 and Task 2 runs captured no raw training logs.** Her notebooks did not write them and they cannot be reconstructed after the fact. Her Task 3 log is present. Her manifests record this.

- **Yashashree’s Task 2 outputs folder is empty**, so there are no loss curves or confusion matrices for her three classifiers.
- **Yashashree’s Task 2 calibration numbers are invalid**, as set out in section 2.3.3, and need recomputing.
- **Anees’s Task 3 checkpoint carries a stale config field** recording nine residual blocks where the weights are unmistakably six at 80 filters. The YAML config and all write ups match the actual weights.
- **Neither member ran the clean identity weight ablation** that Task 3 needs for a proper attribution, because it did not fit the booked GPU time.

CONTENTS

TABLE XXXI

Section	Title
I	Task 1: Character Level Language Model
I.A	Objective
I.B	Member 1: Anees Saheba Guddi
I.C	Member 2: Yashashree Shinde
I.D	Side by Side Comparison
I.E	Joint Analysis
II	Task 2: Yelp Polarity Sentiment Classification
II.A	Objective
II.B	Member 1: Anees Saheba Guddi
II.C	Member 2: Yashashree Shinde
II.D	Side by Side Comparison
II.E	Joint Analysis
III	Task 3: CycleGAN Monet and Photo Style Transfer
III.A	Objective
III.B	Member 1: Anees Saheba Guddi
III.C	Member 2: Yashashree Shinde
III.D	Human Audit
III.E	Side by Side Comparison
III.F	Joint Analysis
IV	Evidence Trail
V	Known Gaps
	References

REFERENCES

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30. [2] Eldan, R. and Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759. [3] Zhu, J.-Y., Park, T., Isola, P. and Efros, A. A. (2017). Unpaired Image to Image Translation using Cycle Consistent Adversarial Networks. IEEE International Conference on Computer Vision. [4] Naeem, M. F., Oh, S. J., Uh, Y., Choi, Y. and Yoo, J. (2020). Reliable Fidelity and Diversity Metrics for Generative Models. International Conference on Machine Learning. Source of the density and coverage metrics used in Task 3.